

HW1: Real-time Analysis of a HW-SW Platform

Lin Yen-Yu
112550098

Abstract—This work analyzes the performance of the Aquila RISC-V core running the CoreMark benchmark using a modular hardware profiler. The profiler measures per-function cycles and classifies them into compute, memory, and stall components, revealing major bottlenecks from data memory (DM) and execution (EX) stalls. Replacing the original 3-cycle multiplier with a DSP-based single-cycle version reduces EX stalls by 28.3%, improving total cycles by 13% and compute efficiency by 14%. The results show that fine-grained hardware profiling effectively identifies microarchitectural inefficiencies and guides targeted performance optimizations on FPGA-based RISC-V systems.

Index Terms—RISC-V, Aquila, FPGA, hardware profiling, CoreMark, performance optimization

I. INTRODUCTION AND ENVIRONMENT

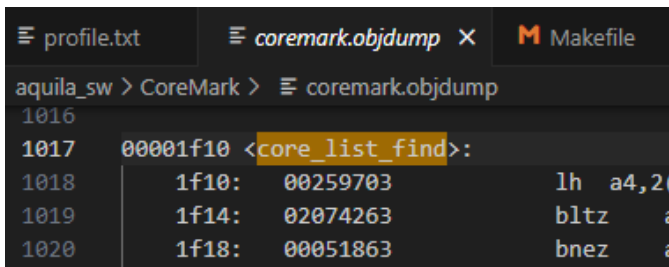
In this assignment, we use the CoreMark CPU benchmark to evaluate the Aquila core. The goal is to identify the top five hotspots in CoreMark when executed on Aquila, and to extend the core with hardware profiling circuits in order to analyze why these functions become hotspots. Based on this profiling, we further classify and examine different sources of stalls, and finally propose architectural improvements.

The experimental environment is **Ubuntu 24.04** with **GCC 13.3.0**. The ELF binaries are generated and transferred via a Makefile, then executed on the FPGA through UART using Tera Term. In parallel, Vivado's Integrated Logic Analyzer (ILA) is employed to monitor the execution waveforms and capture detailed cycle-level information for analysis.

II. PROFILING MECHANISM

A. Finding Hotspots' address

After generating the ELF file, I examined the .objdump output to obtain the start and end addresses of each function. These addresses are essential for mapping the program counter (PC) ranges monitored by the profiler.



```
profile.txt  coremark.objdump x  Makefile
aquila_sw > CoreMark > coremark.objdump
1016
1017 00001f10 <core_list_find>:
1018 1f10: 00259703 lh a4,2(
1019 1f14: 02074263 bltz a
1020 1f18: 00051863 bnez a
```

Fig. 1: Finding Address from .objdump

B. Modular Profiler Design

Since the profiling logic is identical across functions, I adopted a modularized design. I implemented a parameterized profiler module that accepts configurable function address ranges as inputs. Each function's flag signal triggers its own counter. Multiple instances of this module can be instantiated for different functions, and the results are later retrieved via the Integrated Logic Analyzer (ILA).

C. Profiler Implementation

1) *Program Counter Source*: According to the homework specification (page 26), the `exe2mem_pc` signal was used as the reference program counter (PC). Other stages such as decode were not chosen, because pipeline flushes could otherwise lead to incorrect cycle counts.

2) *Memory Access Classification*:

- `exe_re` is used to detect load cycles.
- `exe_we` is used to detect store cycles.
- By analyzing the FSM of the data memory (DMEM), three states were identified: `d_IDLE`, `d_WAIT`, and `d_STALL`.
 - `mem_issue`: the cycle when a load/store request is issued, corresponding to the FSM transition from `d_IDLE` to `d_WAIT`.
 - `mem_wait`: cycles where the request remains pending in `d_WAIT` until `data_ready_i` is asserted.
 - `mem_union`: the union of the two conditions above.

Since the FSM transition to `d_WAIT` is decided in the previous cycle based on `exe_re/exe_we`, a direct classification in the same cycle as `mem_issue` may lead to misalignment. To ensure correctness, both `exe_re` and `exe_we` are registered by one cycle and aligned with `mem_issue`, so that each request can be accurately attributed as either a load or a store. Furthermore, a small sequential block inside the profiler was designed to capture transitions such as “previous cycle = `d_IDLE`, next cycle = `d_WAIT`,” ensuring precise detection of memory issue events.

3) *Stall Classification*: The following stall signals were monitored:

- `stall_pipeline`
- `stall_data_fetch`
- `stall_instr_fetch`
- `stall_from_exe`

By prioritizing them in the order Data Memory → Instruction Fetch → Execution → Hazard, the profiler can attribute stalls to the correct source.

4) *Instruction Issue Classification*: Lastly, the profiler also samples `valid_issue` to classify each issued instruction into five categories for CoreMark on Aquila:

- **Load** (lw, etc.)
- **Store** (sw, etc.)
- **Branch** (conditional branches)
- **Jump** (jal/jalr)
- **ALU** (arithmetic/logic instructions)

III. PROFILING RESULTS

From the profiling data, several observations can be made:

Function	on software (%)	on board (%)
core_list_find()	28.21	13.84
core_list_reverse()	19.04	9.38
core_state_transition()	10.72	18.74
matrix_mul_matrix_bitextract()	8.74	18.01
crcu8()	7.48	9.44
others	25.81	30.59
sum	100.00	100.00

TABLE I: Clock cycles spent on each function (software profiling vs. FPGA board results)

Function	cr_list_find()	cr_list_rev()	cr_st_trans()	mat_mul_mat...()	crcu8()
ALU (%)	0.17	55.38	47.70	87.57	96.43
Branch (%)	17.13	13.43	18.28	3.40	0.00
Jump (%)	0.14	0.23	0.69	0.37	0.00
Load (%)	0.14	13.74	6.06	6.82	1.30
Store (%)	0.00	13.74	2.65	0.38	0.00
IPC	0.18	0.97	0.75	0.98	0.97

TABLE II: Instruction issue breakdown and IPC for each CoreMark hotspot function on Aquila

- **ALU instruction ratio vs. software runtime**: From Table I, it can be seen that functions with higher ALU instruction ratios tend to exhibit shorter execution times in software. For example, `core_list_find()` contains only about 0.17% ALU instructions, while `crcu8()` reaches as high as 96%. This phenomenon can be attributed to the architectural differences between the host CPU and the Aquila 5-stage pipeline CPU. My laptop CPU (Intel Core i7-13700HX) benefits from a large cache hierarchy and hardware multi-threading, which significantly accelerate arithmetic instructions. As a result, the relative cost of ALU operations in the software environment is much lower compared to Aquila.

Function	cr_list_find()	cr_list_rev()	cr_st_trans()	mat_mul_mat...()	crcu8()
compute	73.35%	72.53%	86.32%	92.79%	98.70%
mem issue	26.65%	27.47%	13.68%	7.21%	1.30%
mem wait	0.00%	0.00%	0.00%	0.00%	0.00%
load	26.65%	13.74%	9.81%	6.82%	1.30%
store	0.00%	13.74%	3.87%	0.38%	0.00%

TABLE III: Computation vs. memory cycle breakdown for each CoreMark hotspot function on Aquila

- **Zero cycles in mem_wait**: The profiling results show that the cycle count for `mem_wait` is consistently zero. This indicates that Aquila requires only a single cycle to handle the request and fetch the data, so the data

memory (DMEM) FSM does not remain in the `d_WAIT` or `d_STALL` states.

Function	cr_list_find()	cr_list_rev()	cr_st_trans()	mat_mul_mat...()	crcu8()
stall rate (%)	26.65	27.47	13.68	53.89	1.30
DM (% of stalls)	100.00	100.00	100.00	13.37	100.00
IF (% of stalls)	0.00	0.00	0.00	0.00	0.00
HZ (% of stalls)	0.00	0.00	0.00	0.00	0.00
EX (% of stalls)	0.00	0.00	0.00	86.63	0.00

TABLE IV: Stall breakdown for each CoreMark hotspot on Aquila (overall stall rate and share by source)

- **Data memory (DM) as main stall source**:

For `core_list_find`, `core_list_reverse`, and `core_state_transition`, stall rates are $\approx 27\%$, 28% , and 13.7% , all caused by DM accesses. No IF or hazard stalls were observed.

- **Execute (EX) stalls in matrix_mul**:

`matrix_mul_matrix_bitextract` shows the highest stall rate ($\sim 53.8\%$), with $\sim 86.7\%$ from EX. DM stalls are negligible.

IV. HOW TO IMPROVE AQUILA

A. DMEM FSM Modification

To reduce data-memory stalls, I modified the DMEM FSM so that `d_IDLE` directly returns to `d_IDLE` when `data_ready_i` is valid in the same cycle:

```
d_nxt = data_ready_i ? d_IDLE : d_WAIT;
```

However, no improvement was observed. This indicates that `data_ready_i` is not asserted within the same cycle as the request, and the DMEM requires at least one-cycle latency.

B. Improving Multiplier in muldiv.v

The function `matrix_mul_matrix_bitextract()` suffers from EX stalls due to the 3-cycle multiplier. I replaced it with a single-cycle DSP-based multiplier:

```
(* use_dsp = "yes" *)
wire [W-1:0] fast_result = op_a * op_b;
```

Below are my experimental results:

Cycle count	mat_mul_mat...()	mat_mul_mat()...(M)
total	265,959,440	230,438,560
compute	246,789,400	211,268,520
memory	19,170,040	19,170,040
stall total	143,333,680	108,178,240
stall from DM	19,170,040	19,170,040
stall from EX	124,163,640	89,008,200

TABLE V: Clock cycles of total, compute, memory, and stall parts for `mat_mul_mat()` and `mat_mul_mat() (M)`.

It can be observed that the **stall from EX** has significantly decreased by **28.3%**. The **total cycle count** shows an improvement of nearly **13%**, and the **compute cycle** has also improved by approximately **14%**.

REFERENCES

- [1] <https://www.intel.com/content/www/tw/zh/products/sku/232166/intel-core-i713700hx-processor-30m-cache-up-to-5-00-ghz/specifications.html>
- [2] https://docs.amd.com/r/en-US/ug912-vivado-properties/USE_DSP